DAY 16 | HANDS-ON ACTIVITIES

PySpark Batch Pipelines and SQL: Problem Statements

Seven problems: SQL first, then a production-style Spark pipeline. SQLite and Spark both run in Colab and VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: SQL Warm-up

Level: Warm-up | **Concepts:** SELECT, WHERE, ORDER BY

In SQLite, list orders over 500 sorted by amount descending, then the 3 most recent orders.

Sample run / expected result

```
order_id | city | amount
1008 | Tokyo | 3300.0
...
```

Hints

- ORDER BY amount DESC; ORDER BY order_date DESC LIMIT 3

Problem 2: Groups and HAVING

Level: Easy | **Concepts:** GROUP BY, HAVING

Show revenue and order count per city, only for cities with revenue above 1000.

Sample run / expected result

```
Tokyo | 2 | 4500.0
Bengaluru | 2 | 3350.5
```

Hints

- HAVING SUM(amount) > 1000

Problem 3: Join and Segment

Level: Medium | **Concepts:** LEFT JOIN, COALESCE

Report revenue per customer segment, showing orders with unknown customers as segment "unknown".

Sample run / expected result

```
enterprise | 5060.0
retail | 4262.25
unknown | 930.0
```

Hints

- COALESCE(c.segment, 'unknown')

Problem 4: CTE and Window

Level: Medium | **Concepts:** WITH, RANK() OVER

Using a CTE for daily city revenue, rank cities within each day and return only rank 1 per day.

Sample run / expected result

```
2026-11-12 | Tokyo | 4500.0
2026-11-13 | Bengaluru | 3100.0
```

Hints

- RANK() OVER (PARTITION BY order_date ORDER BY revenue DESC)

Problem 5: Same Query in Spark SQL

Level: Medium | **Concepts:** temp views, spark.sql

Register orders and customers as temp views and run your CTE and window query with spark.sql. Confirm the results match SQLite.

Sample run / expected result

```
Spark result == SQLite result: True
```

Hints

- Convert both to sorted lists of tuples to compare.

Problem 6: Parameterised Pipeline

Level: Challenging | **Concepts:** functions per layer, run date, run log

Write run(run_date) that processes only that day's orders through silver and gold, writes the gold partition with overwrite, and appends a row to a run log (date, rows in, rows out, duration).

Sample run / expected result

```
run("2026-11-13") -> bronze 3, silver 3, gold 3
Re-run -> same counts, log has 2 rows
```

Hints

- Filter bronze by order_date == run_date first.

Problem 7: Quality Gates and Tests

Level: Stretch | **Concepts:** validation, PyTest

Add checks that stop the run if silver is empty or has duplicate keys, and a reconciliation check that gold revenue equals silver revenue. Write PyTest tests for each check.

Sample run / expected result

```
check_unique raised PipelineError: duplicate order_id
5 passed
```

Hints

- class PipelineError(Exception): pass

Ganbatte (頑張って): do your best